

OpenClaw Security Risks in Setup and Usage

Executive summary

Data collection date: March 15, 2026 (America/Los_Angeles). ¹

OpenClaw is a locally run “agent platform” that can execute shell commands, read/write files, send messages across chat channels, fetch arbitrary URLs, schedule tasks, and access connected services—i.e., it behaves like a high-privilege automation runtime, not a normal chatbot. ²

The dominant sources of real-world risk cluster into four buckets:

- **Local browser-to-agent attacks (loopback isn’t a safety boundary).** Multiple disclosed issues show that a user merely visiting a malicious web page can cause credential exposure or enable authenticated control of a local gateway under reachable conditions (for example, the token-exfiltration “gatewayUrl” class, and “browser-origin WebSocket auth hardening gap” chains). ³
- **Remote exposure via misconfiguration (reverse proxies, “trusted-proxy” mode, public templates).** Several advisories and community reports highlight that “localhost trust assumptions” can become dangerous once you add reverse proxies, remote access, or cloud hosting; specific vulnerabilities in `trusted-proxy` mode and default assumptions have led to privileged access paths. ⁴
- **Supply-chain risk through skills/plugins.** OpenClaw’s skill ecosystem is explicitly treated as untrusted code by OpenClaw’s own documentation; public reporting identified malware embedded in marketplace skills, prompting OpenClaw to add automated marketplace scanning through VirusTotal ⁵ and additional hardening measures. ⁶
- **Rapid change velocity and advisory volume.** The core repository shows extremely high churn (tens of thousands of commits), frequent releases/tags, and a large and fast-moving advisory stream—meaning “time-to-patch” and configuration drift are persistent operational risks. ⁷

Strategic conclusion: treat an OpenClaw deployment like you would treat a **privileged remote administration plane** (operator credential, remote execution surface, high-value secrets store). OpenClaw documentation explicitly frames the gateway as a control plane and warns that several mechanisms (for example exec approvals) are not a substitute for hostile multi-tenant isolation. ⁸

What OpenClaw is and how it’s built

OpenClaw’s own project description frames it as an “open agent platform” running on your infrastructure and reachable via chat apps (WhatsApp, Telegram, Discord, Slack, Teams, etc.), with local control and locally held keys/data. ⁹

Core components and trust boundaries

OpenClaw security documentation and the Trust program define a model with distinct surfaces: ¹⁰

- **Gateway:** the control plane and policy enforcement surface (authentication, tool policy, routing, session selection). ⁸
- **Nodes:** paired remote execution surfaces tied to a gateway; after pairing, node actions are treated as operator actions on that node (commands, device capabilities, etc.). ¹¹
- **Control UI / Web chat:** browser client(s) that connect over WebSocket; the UI can run from different HTTP origins and target different gateway WS URLs (a flexibility that has security consequences when origin validation is wrong or bypassed). ¹²
- **HTTP endpoints:** optional OpenResponses-compatible `POST /v1/responses` and other “invoke” endpoints; tokens/passwords for these endpoints should be treated as full operator credentials (not fine-grained per-user keys). ¹³
- **Tools:** explicit capabilities (filesystem, exec, browser/canvas, messaging, cron, nodes, etc.) subject to layered policy (global/provider/agent/sandbox/subagent). ¹⁴
- **Sandboxing / elevated mode:** Docker-based isolation is available; default Docker sandbox networking is `none` (no egress), and “elevated” is an explicit escape hatch to run host exec. ¹⁵
- **Skills and plugins:** skills can be installed from a public registry, bundled with plugins, or discovered in workspaces; docs explicitly say to treat third-party skills as untrusted and note that some secret injection occurs in the host process, not the sandbox. ¹⁶

A key architectural point: OpenClaw’s docs explicitly warn against misinterpreting safety features—exec approvals are “guardrails for operator intent,” not a strong isolation boundary, and hostile-user isolation requires splitting trust boundaries (separate OS users/hosts/gateways). ¹⁷

High-value security implication

Because OpenClaw centralizes messaging identities, automation, secrets, and execution, an attacker who obtains operator-level gateway access can often pivot into: reading session transcripts, extracting configured tokens/keys, manipulating tool policy/sandbox settings, triggering actions via tools/channels, and (depending on configuration) reaching command execution. ¹⁸

Release and development cadence

Timeline and naming history

OpenClaw’s official blog describes the project’s naming path: an earlier name “Clawd” in November 2025 (a “Claude” pun), then “Moltbot,” and finally OpenClaw announced January 29, 2026, after trademark checks and migration work. ¹⁹

The same announcement claims the project surpassed 100,000 GitHub stars rapidly and notes a security-focused burst (“34 security-related commits”) alongside feature additions (new channels/plugins, new model support, image support in web chat). ¹⁹

Observable cadence signals

The main repo shows very high churn (19,071 commits at the time of capture) and a large monorepo/tooling surface (for example secret-scanning configuration files and pre-commit config present in the repo root). ²⁰

Release tags visible in March 2026 show date-stamped versions across consecutive days (for example tags like `v2026.3.13`, `v2026.3.12`, `v2026.3.11`) suggesting near-daily publishing. ²¹

Notable recent changes that materially affect security posture

OpenClaw's Trust site describes a formal security program with explicit phases (threat model transparency, public roadmap, code review, and triage SLAs), and it enumerates "secure by default" controls such as pairing for unknown senders, deny-by-default exec behavior, default "self-only" allowFrom behavior, session isolation, SSRF protections, and default gateway auth requirements. ²²

The project also announced adding marketplace scanning and daily rescans for skills published to its registry (including deterministic packaging, SHA-256 hashing, and automated verdicting plus blocking). ²³

Important operational implication: because releases happen quickly and advisories are frequent, "upgrade lag" is itself a vulnerability class (windows of exposure repeat). ²⁴

Vulnerabilities and advisories

Method and caveats

This section prioritizes: repo security advisories (GHSA), the GitHub ²⁵ advisory database, CVE/NVD records, and OpenClaw's official Trust/docs. ²⁶

Caveats that materially affect completeness/precision:

- Some NVD entries are "undergoing analysis" or lack NIST-scored CVSS, while GitHub advisories often contain the most immediately actionable version ranges and fix commits. ²⁷
- Some issues are GHSA-only ("No known CVE"), and the project is publishing advisories at a pace that makes "complete enumeration" time-sensitive; this list reflects what was directly retrievable by primary sources during this collection window. ²⁸
- OpenClaw's own security guidance discourages treating "prompt injection alone" as a product vulnerability in its trust model unless it crosses policy/auth boundaries; some third-party CVE claims may be disputed or reclassified. ²⁹

High-impact CVEs in the last 12 months

Table notes: "Exploitability" is described as conditions and attacker position implied by the advisory and CVSS vectors; "primary source" links are via citations.

ID	Published date	Severity	Affected versions	Exploitability and prerequisites	Impact	Fixed / patched
CVE-2026-25253 (GHSA-g8p2-7wf7-98mq)	2026-02-01	CVSS 8.8 High	< 2026.1.29	Victim visits crafted link/site; token exfiltration via unvalidated <code>gatewayUrl</code> and auto-connect; works even if gateway binds to loopback because browser bridges. ³⁰	Full gateway compromise (config changes, tool/sandbox changes, privileged actions → practical RCE depending on enabled tools). ³¹	2026.1.29 ³⁰
CVE-2026-24763 (GHSA-mc68-q9jw-2h3v)	2026-02-02	CVSS v3.1 vector indicates High	< 2026.1.29	Authenticated user able to influence env vars; command injection in Docker sandbox exec due to unsafe PATH handling. ³²	In-container command injection; can undermine sandbox expectations. ³³	2026.1.29 ³²
CVE-2026-25157	2026-02-04	High	< 2026.1.29	OS command injection through SSH node helper script when user-supplied project paths are interpolated into shell without escaping; affects remote SSH host targeted by node command. ³⁴	Arbitrary command execution on the SSH target host in the affected path. ³⁴	2026.1.29 ³⁴
CVE-2026-25593 (GHSA-g55j-c2v4-pjcg)	2026-02-06	CVSS v3.1 8.4 High	< 2026.1.20	Unauthenticated local client can call WebSocket API <code>config.apply</code> to set unsafe <code>cliPath</code> , enabling command injection as gateway user. ³⁵	Local privilege execution as gateway process user; can lead to full host compromise in multi-user machines. ³⁶	2026.1.20 ³⁵

ID	Published date	Severity	Affected versions	Exploitability and prerequisites	Impact	Fixed / patched
CVE-2026-25475 (GHSA-r8g4-86fx-92mq)	2026-02-04	High	≤ 2026.1.30	Agent can read arbitrary files by emitting <code>MEDIA:/path/to/file</code> ; file path validation allows absolute/traversal paths. ³⁷	Sensitive file disclosure/exfil to user/channel (secrets, credentials). ³⁸	2026.1.30 ³⁷
CVE-2026-28472 (GHSA-rv39-79c4-7459)	2026-03-05	Critical per CVSS vector	< 2026.2.2	WebSocket connect handshake could skip device identity checks based on presence of token value rather than validated auth; affects reachable deployments. ³⁹	Potential operator access without device identity/pairing under specific configs; crosses intended auth boundary. ³⁹	2026.2.2 ³⁹
CVE-2026-28470 (GHSA-3hcm-ggvf-rch5)	2026-03-05	Critical per CVSS v3.1 vector	< 2026.2.2	If exec approvals allowlist mode is enabled, command substitution inside quotes (<code>\$()</code> / backticks) can bypass allowlist. ⁴⁰	Approval/allowlist bypass leading to command execution outside intended policy. ⁴¹	2026.2.2 ⁴²
CVE-2026-28363 (GHSA-3c6h-g97w-fg78)	2026-02-26	High/Critical (CVSS shown as 8.8 on GHSA; v3.1 vector indicates high)	< 2026.2.23	In allowlist mode, <code>tools.exec.safeBins</code> validation for <code>sort</code> bypassable via GNU long-option abbreviations (approval skipped). ⁴³	Approval-free execution paths intended to require approval; undermines “safeBins” policy intent. ⁴⁴	2026.2.23 ⁴³

ID	Published date	Severity	Affected versions	Exploitability and prerequisites	Impact	Fixed / patched
CVE-2026-28485	2026-03-05	High (v3.1 vector)	2026.1.5–<2026.2.12	Missing mandatory auth on <code>/agent/act</code> browser-control route; local or LAN callers can invoke privileged operations. ⁴⁵	Unauthorized invocation of browser-context actions; access to sensitive in-session data. ⁴⁵	2026.2.12 ⁴⁵
CVE-2026-28458 (GHSA-mr32-vwc2-5j6h)	2026-03-05	High (v3.1 8.1 shown in CVE record sources; NVD includes description)	2026.1.20–<2026.2.1	If Chrome extension relay is enabled: <code>/cdp</code> WebSocket endpoint lacks auth; websites can reach loopback and access sensitive browser context via CDP. ⁴⁶	Cross-tab cookie access and JS execution in other tabs (browser compromise path). ⁴⁷	2026.2.1 ⁴⁸
CVE-2026-27488	2026-02-21	SSRF (severity depends on deployment)	≤ 2026.2.17	Cron webhook delivery used <code>fetch()</code> directly; attacker-controlled webhook targets can reach private/metadata/internal endpoints without SSRF policy checks. ⁴⁹	SSRF to internal networks / metadata endpoints from the gateway. ⁵⁰	2026.2.19 ⁵¹
CVE-2026-32302 (GHSA-5wcw-8jjv-m286)	2026-03-12 (CVE record)	High (CVSS 8.1 in CNA record)	< 2026.3.11	In <code>trusted-proxy</code> mode, browser-origin WebSocket could bypass origin validation through trusted proxy headers; affects reverse-proxy deployments relying on allowed-origins restrictions. ⁵²	Privileged operator session from untrusted browser origin; verified impact includes sensitive config reads. ⁵³	2026.3.11 ⁵³

Notable high/critical GHSA issues without known CVEs

These are examples of security advisories released in March 2026 that explicitly state “No known CVE” but still have maximum operational impact when applicable. ⁵⁴

GHSA	Published	Severity	Affected → fixed	Why it matters
GHSA-rqpp-rjj8-7wv8	2026-03-13	Critical (CVSS 10.0)	≤ 2026.3.11 → 2026.3.12	Shared-authenticated WebSocket clients could self-declare elevated scopes like <code>operator.admin</code> without server binding, enabling admin-only gateway operations. ⁵⁵
GHSA-5wcw-8jjv-m286	2026-03-12	High	< 2026.3.11 → 2026.3.11	Browser-origin WS origin validation bypass in <code>trusted-proxy</code> mode (reverse proxy risk). ⁵⁶
GHSA-v773-r54f-q32w	2026-02-15	Moderate	≤ 2026.2.13 → 2026.2.14	Slack <code>dmPolicy=open</code> let any DM sender run privileged slash commands (authorization boundary failure). ⁵⁷
GHSA-jmmg-jqc7-5qf4	2026-02-26 (published to advisory DB Mar 3)	Moderate (but exploitable chains described)	≤ 2026.2.24 → 2026.2.25	Browser-origin WebSocket auth hardening gap: origin checks + loopback throttling + silent pairing combined into a password-guess chain on loopback password deployments. ⁵⁸

Vulnerability themes that repeat

The advisories and OpenClaw’s own threat modeling show recurring patterns, important for “future-proof” hardening: ⁵⁹

- **Auth boundary mistakes in “local trusted” paths** (loopback exemptions, implicit trust of local origin, silent pairing flows). ⁶⁰
- **Policy-analysis/runtime-execution mismatches** in allowlist/approval systems (shell parsing, long-option abbreviations, command substitution). ⁶¹
- **SSRF and URL handling via webhooks or file/image URL inputs** (guarded fetch inconsistencies, cron webhook SSRF, URL-based file/image ingestion). ⁶²
- **File/path boundary bugs** (path traversal, symlink escapes, unsafe media path handling). ⁶³
- **Denial-of-service via unbounded parsing/allocations** (oversized payloads, base64 decode before enforcing limits). ⁶⁴

Deployment risks, misconfigurations, and supply-chain exposure

Common insecure deployment patterns

The following issues are repeatedly called out across OpenClaw's own docs, advisories, and community reports.

Exposing the gateway beyond a trusted network boundary

OpenClaw's own docs emphasize that gateway auth and "sessionKey" have different meanings, and that gateway credentials represent trusted operator scope; the OpenResponses HTTP API specifically states a valid token/password should be treated as an owner/operator credential. ⁶⁵
Despite that, external templates and real-world setups sometimes expose gateways publicly with insufficient outer access control. ⁶⁶

Reverse proxy mistakes and `trusted-proxy` foot-guns

Community and issue reports describe a failure mode: treating "localhost connections are trusted" as safe, then placing a reverse proxy in front such that external traffic appears to OpenClaw as trusted-local/proxied traffic, bypassing expected auth controls. ⁶⁷
OpenClaw's `trusted-proxy` documentation explicitly warns that in this mode, Control UI sessions can connect without device pairing identity, making your reverse proxy policy the effective access control, and requiring strict trusted-proxy IP locking and firewalling. ⁶⁸

Using password auth on loopback without accounting for browser-origin WebSocket behavior

A major theme in February 2026 research: a malicious site can initiate WebSocket connections to `localhost` from within the user's browser, and under certain OpenClaw versions/configs, brute-force and silent pairing behaviors were possible. ⁶⁹
OpenClaw patched several of these conditions (origin checks, loopback throttling, silent pairing restrictions) in the `>=2026.2.25` window per advisory. ⁷⁰

Over-broad tool enablement and missing "deny first" posture

OpenClaw supports layered tool allow/deny policies and group shorthands; a "deny wins" model is explicitly documented, and guidance includes denying `exec` for untrusted agents and disabling elevated mode globally when you only want sandboxed execution. ⁷¹
OpenClaw's security guidance also makes clear that "exec approvals" are not a substitute for sandboxing/host isolation when hostile inputs/users are in scope. ⁷²

Third-party integrations and supply-chain risks

Skills marketplace and untrusted skills

OpenClaw docs explicitly instruct: treat third-party skills as untrusted code, read before enabling, prefer sandboxing for risky tools/inputs, and understand that some secrets are injected into the host process during an agent turn (not the sandbox). ⁷³

Public reporting in early February 2026 described malware embedded in skills distributed via the public registry, used as a delivery vehicle for credential stealers and other payloads. ⁷⁴

In response, OpenClaw announced a partnership with VirusTotal to scan skill bundles on publication and re-scan daily, using deterministic packaging + SHA-256 hashing, and blocking downloads for “malicious” verdicts. ²³

Plugins and workspace-based discovery

Skills can be shipped via plugins, and plugins can be discovered/enabled in ways that participate in skill precedence rules. ⁷⁵

The advisory list includes issues where workspace plugin auto-discovery can become code execution from cloned repositories if not carefully constrained—highlighting that “developer convenience” features can become supply-chain ingress when workspaces contain untrusted code. ⁷⁶

Provider/API integrations and secrets

The Trust program and docs enumerate that OpenClaw commonly holds API keys and connected service credentials, and that it can send messages as you—so credential scope and rotation matter as much as patching. ⁷⁷

Threat models and attack scenarios

OpenClaw’s Trust threat model is based on the MITRE ATLAS framework and lists multiple critical and high-risk threats, including malicious skills as entry points, direct and indirect prompt injection, token theft, and “prompt injection to RCE” patterns when approvals/policies are bypassed. ⁷⁸

Below are practical threat models mapped to common environments.

Local workstation threat model

Primary risks:

- **Browser-to-local-gateway bridging** (cross-origin WebSocket to loopback) enabling token theft or password-auth chains under vulnerable releases or weak configurations. ⁷⁹
- **Local co-tenant process attacks** (another process on the same machine) abusing local-only APIs or configuration surfaces in vulnerable versions (for example unauth local WebSocket config write → command injection). ⁸⁰
- **Secret exfil via agent behaviors** (file reads, media path tricks, transcript export) if policies allow it or if vulnerabilities permit it. ⁸¹

Network and reverse-proxy threat model

Primary risks:

- **Reverse proxy misbinding** and `trusted-proxy` origin/auth mistakes leading to privileged operator sessions from untrusted browser origins. ⁸²

- **Direct network reachability of the gateway** (LAN/VPN/WAN) turning configuration bugs into remote compromise. ⁸³

Cloud / VPS threat model

Primary risks:

- **Internet exposure by default** (templates, “quick deploy” guides) plus rapid patch cadence, leading to drift and unpatched exposure windows. ⁸⁴
- **Cloud metadata SSRF** via webhook/URL fetch paths if SSRF guards are bypassed or inconsistent, enabling credential theft from instance metadata endpoints. ⁸⁵

CI/CD usage threat model

OpenClaw’s own security program scope explicitly includes “build and release pipeline” and dependency auditing, signed releases, and supply-chain controls as defensive engineering goals. ⁸⁶

If OpenClaw (or skills/plugins) are used in CI/CD contexts, threats broaden to: secret exposure in logs/prompts, malicious dependency updates, or executing untrusted code with CI runner credentials (especially if “elevated” execution is enabled). ⁸⁷

Mermaid attack-path diagrams

Token exfiltration to gateway takeover (representative of CVE-2026-25253 class)

```

flowchart LR
    A[User visits crafted URL / malicious site] --> B[Control UI uses unvalidated gatewayUrl]
    B --> C[Browser auto-connects via WebSocket]
    C --> D[Auth token sent to attacker-controlled endpoint]
    D --> E[Attacker connects to victim gateway as operator]
    E --> F[Change config: tools/sandbox/policies]
    F --> G[Invoke privileged actions]
    G --> H[Potential code execution / data theft]

```

This chain is explicitly described as token exfiltration that leads to full gateway compromise and practical 1-click RCE, including the “works on loopback binds” property. ⁸⁸

Reverse-proxy trusted-proxy origin bypass (CVE-2026-32302 / GHSA-5wcw-8jjv-m286)

```

flowchart LR
    A[Gateway behind trusted reverse proxy] --> B[trusted-proxy mode enables proxy identity headers]
    X[Victim loads attacker page on untrusted origin] --> C[Browser opens cross-site WebSocket to proxy endpoint]
    C --> D[Origin validation bypass when proxy headers are present]
    D --> E[Browser inherits proxy-authenticated identity]

```

```
E --> F[Privileged operator session established]
F --> G[Sensitive reads (e.g., config) / privileged calls]
```

OpenClaw’s advisory describes this as a browser-originated WebSocket bypass in `trusted-proxy` mode that can establish privileged operator sessions and reach sensitive methods like `config.get`. ⁵²

Detection, hardening, and remediation roadmap

Detection and monitoring recommendations

OpenClaw’s Trust posture recommends an explicit verification command (`openclaw security audit --deep`) and highlights key items like DM policy, allowFrom, exec security, gateway binding, and workspace secrets hygiene. ⁸⁹

Build an operational monitoring posture around the following evidence sources:

- **Gateway access logs and auth failure telemetry:** watch for bursts of authentication failures from loopback/browser-origin classes (especially relevant to WebSocket brute-force chains that were fixed in later versions). ⁵⁸
- **Config mutation monitoring:** alert on changes to gateway auth mode, trusted proxy settings, tool policy, sandbox settings, and any “dangerously*” or break-glass toggles. ⁹⁰
- **Skill install/update events:** treat skill updates as supply-chain events; log skill hashes/versions and reconcile to expected registries; OpenClaw’s own marketplace scanning is explicitly “not a silver bullet.” ⁹¹
- **Outbound network monitoring** from gateway and sandbox containers: SSRF and exfiltration patterns (metadata IPs, unusual domains) align to the threat model and past SSRF advisories. ⁹²
- **File permission checks** for transcripts and workspace artifacts, given the existence of advisories around transcript file permission hardening. ⁹³

Incident response: minimum viable playbook

Because gateway tokens/passwords are “operator credentials,” treat any suspicion of token theft or unauthorized WebSocket session establishment as a full credential-compromise incident. ⁹⁴

Core steps (ordered):

1. **Isolate the gateway host** from untrusted networks (remove public ingress; restrict to loopback/VPN only). ⁹⁵
2. **Rotate all gateway auth secrets** (token/password) and revoke/rotate connected provider keys (model APIs, bot tokens, webhook secrets). ⁹⁶
3. **Audit recent config changes** (auth mode flips, trusted-proxy settings, tool/sandbox policy changes). ⁹⁷
4. **Review sessions/logs for unauthorized actions**, including message sending, file reads, and tool invocations. ⁸⁶
5. **Enumerate and re-validate installed skills/plugins**, comparing to known-good sources and version pins; treat any unexpected skill as potential malware. ⁹⁸

Concrete mitigation steps and secure configuration checklist

This checklist is aligned to OpenClaw's own security documentation, Trust guidance, and the failure modes seen in CVEs/GHSAs.

Network exposure and access control

- Keep the gateway bound to loopback unless you have a controlled network boundary (VPN/tailnet) and strong auth. ⁹⁹
- Prefer HTTPS via Tailscale Serve; if using tokenless Serve, understand it assumes the gateway host is trusted, and disable `gateway.auth.allowTailscale` when untrusted local code could run. ¹⁰⁰
- If using `trusted-proxy` mode, lock ingress to trusted proxy IPs only and treat proxy auth policy as the effective gate (pairing is no longer primary). ⁶⁸
- Do not expose optional HTTP endpoints (OpenResponses, tools invoke) unless required; when enabled, treat the bearer token as full operator access. ¹³

Auth mode selection

- Prefer long, high-entropy token auth over human-chosen passwords for any loopback-exposed WebSocket surface, given documented browser-origin brute-force chain conditions. ¹⁰¹
- After patching, still assume “loopback” is reachable from the browser and therefore not equivalent to “trusted.” ¹⁰²

Tool policy and sandboxing

- Start from a “deny-first” tool policy and selectively allow only required tool groups; OpenClaw documents that deny wins and provides per-agent deny patterns (including denying `exec` for untrusted agents). ¹⁰³
- Prefer sandboxed execution for risky tools/inputs; keep Docker sandbox networking at the secure default (`none`) unless you have a strong reason and compensating controls. ¹⁰⁴
- Disable elevated execution (`tools.elevated.enabled: false`) unless you explicitly need host-level exec escape hatches. ¹⁰⁵
- Assume exec approvals/allowlists can be bypassed by parsing mismatches unless fully patched; multiple advisories/cves exist specifically in this space. ¹⁰⁶

Skills, plugins, and supply chain

- Treat third-party skills as untrusted code and review before enabling; do not store secrets in prompts/logs; understand that `skills.entries.*.env` / `apiKey` inject into host process for the agent turn. ¹⁰⁷
- Maintain an internal allowlist/registry for skills in enterprise settings; do not rely solely on marketplace scanning as it is explicitly not complete. ¹⁰⁸
- Avoid workspace plugin auto-discovery for directories containing untrusted cloned repos; treat it as code execution surface. ⁷⁶

Automated hardening examples

These examples assume you will run OpenClaw as a system service on Linux, binding to loopback, with firewall rules and basic file permission hygiene. Adjust ports/paths to your installation.

Bash hardening script (illustrative)

```
#!/usr/bin/env bash
set -euo pipefail

OPENCLAW_USER="openclaw"
OPENCLAW_HOME="/var/lib/openclaw"
OPENCLAW_WORKSPACE="${OPENCLAW_HOME}/workspace"
OPENCLAW_CONFIG_DIR="/etc/openclaw"
OPENCLAW_PORT="18789"

# 1) Create dedicated user (principle: split trust boundaries by OS user/host)
id -u "${OPENCLAW_USER}" >/dev/null 2>&1 || useradd --system --create-home --
home-dir "${OPENCLAW_HOME}" --shell /usr/sbin/nologin "${OPENCLAW_USER}"

# 2) Create directories with tight perms
install -d -m 0700 -o "${OPENCLAW_USER}" -g "${OPENCLAW_USER}" "${
OPENCLAW_WORKSPACE}"
install -d -m 0750 -o root -g "${OPENCLAW_USER}" "${OPENCLAW_CONFIG_DIR}"

# 3) Generate high-entropy gateway token
GATEWAY_TOKEN="$(openssl rand -hex 32)"
echo "OPENCLAW_GATEWAY_TOKEN=${GATEWAY_TOKEN}" > "${OPENCLAW_CONFIG_DIR}/
secrets.env"
chmod 0640 "${OPENCLAW_CONFIG_DIR}/secrets.env"
chown root:"${OPENCLAW_USER}" "${OPENCLAW_CONFIG_DIR}/secrets.env"

# 4) Minimal config: bind loopback, token auth, conservative defaults
cat > "${OPENCLAW_CONFIG_DIR}/openclaw.json" <<'JSON'
{
  "gateway": {
    "bind": "loopback",
    "port": 18789,
    "auth": { "mode": "token" }
  },
  "tools": {
    "deny": ["group:runtime", "browser", "canvas"],
    "elevated": { "enabled": false }
  },
  "agents": {
    "defaults": {
      "sandbox": { "mode": "non-main", "scope": "session", "workspaceAccess":
```

```

"none" }
    }
  }
}
JSON
chmod 0640 "${OPENCLAW_CONFIG_DIR}/openclaw.json"
chown root:"${OPENCLAW_USER}" "${OPENCLAW_CONFIG_DIR}/openclaw.json"

# 5) Firewall: deny inbound except loopback (example with ufw if present)
if command -v ufw >/dev/null 2>&1; then
    ufw deny "${OPENCLAW_PORT}"/tcp || true
    ufw allow in on lo to 127.0.0.1 port "${OPENCLAW_PORT}" proto tcp || true
fi

echo "Hardened baseline written to ${OPENCLAW_CONFIG_DIR}. Token stored in ${OPENCLAW_CONFIG_DIR}/secrets.env (root:${OPENCLAW_USER}, 0640)."
```

Why these choices are aligned with OpenClaw's threat model: binding to loopback and treating gateway token as full operator credential; using deny-first tool policy; disabling elevated; and preferring sandboxing for non-main sessions match OpenClaw guidance and the threat patterns behind multiple CVEs/GHSAs.

109

Ansible role snippet (illustrative)

```

- name: Create openclaw system user
  ansible.builtin.user:
    name: openclaw
    system: true
    create_home: true
    home: /var/lib/openclaw
    shell: /usr/sbin/nologin

- name: Create config and workspace directories
  ansible.builtin.file:
    path: "{{ item.path }}"
    state: directory
    owner: "{{ item.owner }}"
    group: "{{ item.group }}"
    mode: "{{ item.mode }}"
  loop:
    - { path: "/etc/openclaw", owner: "root", group: "openclaw", mode: "0750" }
    - { path: "/var/lib/openclaw/workspace", owner: "openclaw", group:
"openclaw", mode: "0700" }

- name: Generate gateway token
  ansible.builtin.command: "openssl rand -hex 32"
```

```

register: openclaw_token
changed_when: false

- name: Write secrets env file
  ansible.builtin.copy:
    dest: /etc/openclaw/secrets.env
    content: "OPENCLAW_GATEWAY_TOKEN={{ openclaw_token.stdout }}\n"
    owner: root
    group: openclaw
    mode: "0640"

- name: Deploy openclaw.json
  ansible.builtin.copy:
    dest: /etc/openclaw/openclaw.json
    owner: root
    group: openclaw
    mode: "0640"
    content: |
      {
        "gateway": {
          "bind": "loopback",
          "port": 18789,
          "auth": { "mode": "token" }
        },
        "tools": {
          "deny": ["group:runtime", "browser", "canvas"],
          "elevated": { "enabled": false }
        }
      }

- name: (Optional) UFW deny gateway from non-loopback
  community.general.ufw:
    rule: deny
    port: "18789"
    proto: tcp

```

These tasks reflect OpenClaw's documented need to split trust by OS user/host when hostile-user isolation matters, and to keep gateway ingress constrained. ¹¹⁰

Terraform example (AWS EC2 skeleton)

```

# Illustrative: only allow SSH from a VPN/bastion CIDR; do NOT expose OpenClaw
port publicly.
resource "aws_security_group" "openclaw_host" {
  name      = "openclaw-host"
  description = "EC2 host for OpenClaw (no public gateway ingress)"

```

```

ingress {
  description = "SSH from VPN/bastion"
  from_port   = 22
  to_port     = 22
  protocol    = "tcp"
  cidr_blocks = ["10.0.0.0/16"] # replace with your admin network
}

# No ingress rule for 18789: keep gateway loopback-only, access via SSH
tunnel/VPN.
egress {
  from_port = 0
  to_port   = 0
  protocol  = "-1"
  cidr_blocks = ["0.0.0.0/0"]
}
}

# For remote access, prefer VPN/tailnet; OpenClaw docs show SSH tunnel fallback
to loopback.

```

This matches OpenClaw’s own remote-access guidance: prefer VPN/tailnet; fallback is SSH tunnel to loopback; and even with tunnels, clients must present gateway auth. ¹¹¹

Recommended tools and tests, and how to run them against OpenClaw

OpenClaw’s Trust program explicitly calls for manual review plus “automated scanning” (static analysis, dependency auditing, secret detection) and dynamic testing of documented attack patterns. ⁸⁶ The repository also includes secret-scanning and pre-commit oriented artifacts (indicative of an internal hygiene baseline). ¹¹²

Representative tool categories and practical usage:

Goal	Tooling category	Examples	How to run (conceptual)	What to look for (OpenClaw-specific)
SAST	Static code analysis	CodeQL, Semgrep	Run on repo; focus on gateway WS handlers, auth, tool policy, SSRF utilities	Origin validation, loopback exemptions, scope binding, canonicalization pitfalls ¹¹³
Dependency risk	Dependency auditing	lockfile scanners, GHSA/OSV checks	Audit npm deps for advisories; monitor GHSA stream	Rapid GHSA releases; map GHSA→CVE; ensure deployed version includes fix commits ⁹³

Goal	Tooling category	Examples	How to run (conceptual)	What to look for (OpenClaw-specific)
Secret scanning	Secret detection	git hooks, secret scanners	Scan repo + workspace; monitor logs/config backups	Skills can inject secrets into host process; prevent prompt/log leakage ¹¹⁴
DAST	Runtime scanning	WebSocket/API fuzzing; HTTP endpoint scanning	Test Control UI + WS connect flows, proxy configurations	Cross-origin WS behavior; trusted-proxy flows; auth rate limiting; scope binding ¹¹⁵
Fuzzing	Input fuzzing	protocol fuzzers (WS/JSON), file/PDF parsers	Fuzz file ingestion and guarded fetch	Memory limits enforced before allocation; base64 decode limits; redirect/DNS guard behavior ¹¹⁶

Because OpenClaw evolves daily, tests should be wired into a **continuous validation loop**: pull latest release → run SAST/dependency scan → run a small suite of regression DAST tests for known failure modes (gatewayUrl token exfil class, browser-origin WS auth, trusted-proxy origin enforcement, SSRF guard policies). ¹¹⁷

Prioritized remediation roadmap by risk and effort

This is a pragmatic, defense-in-depth ordering that matches the threat model, the CVE themes, and the highest-impact misconfig patterns.

Immediate actions

- Upgrade to at least the latest patched release line; several critical issues have fixes in March 2026 point releases (for example trusted-proxy origin bypass fixed in 2026.3.11, shared-auth scope bug fixed 2026.3.12). ¹¹⁸
- Remove any public/LAN exposure of the gateway; bind to loopback and require token auth everywhere you can. ¹¹⁹
- Rotate gateway tokens/passwords and connected API keys if any exposure is suspected (the OpenResponses API explicitly treats bearer tokens as owner credentials). ¹²⁰
- Disable optional high-risk surfaces unless needed (browser relay, OpenResponses endpoint, tools invoke endpoints). ¹²¹
- Lock skills: audit installed skills; remove unknown publishers; prefer internal registries; treat “clean scan” signals as non-authoritative. ⁹⁸

Near-term hardening

- Implement deny-first tool policy; disable elevated; enforce sandboxing for non-main sessions and untrusted inputs. ¹²²
- If using reverse proxies, explicitly configure `trustedProxies` and lock ingress; avoid `trusted-proxy` mode unless you can enforce strict origin and proxy boundary controls. ⁶⁸

- Establish monitoring: detect unusual WS connection patterns, config changes, skill updates, and SSRF-like outbound traffic. ¹²³

Medium-term programmatic controls

- Deploy separate gateways per trust domain (per OS user/host) if hostile-user or multi-tenant use is in scope; OpenClaw explicitly recommends splitting trust boundaries rather than expecting per-message isolation in a shared gateway. ¹⁰
- Institutionalize a “skill onboarding” pipeline: code review + reproducible bundle hashing + allowlist distribution, so that the public marketplace is not your production supply chain. ⁹¹

Remediation workflow diagram

```

flowchart TD
    A[Discover deployment inventory] --> B[Check reachability: public/LAN/loopback]
    B --> C[Verify version vs advisories/CVEs]
    C --> D[Patch/upgrade to fixed release]
    D --> E[Rotate gateway secrets + provider tokens]
    E --> F[Harden config: bind, auth mode, proxy, tool policy, sandbox]
    F --> G[Validate with security audit + regression tests]
    G --> H[Continuous monitoring + alerting]
    H --> C
  
```

This loop is consistent with OpenClaw’s own security program emphasis on transparency, automated scanning, dynamic testing, and ongoing auditing/verification. ¹²⁴

Gaps and open research questions

These are areas where primary sources indicate unresolved/structural risk or where the evidence is incomplete/ambiguous:

- **Prompt injection remains explicitly “industry-wide unsolved,”** and OpenClaw’s threat model includes direct and indirect prompt injection chains; the open question is how effective forthcoming defenses will be in practice across heterogeneous channels and skills. ¹²⁵
- **CVE classification boundaries are contentious.** OpenClaw security guidance discourages treating prompt-injection-only chains as product vulnerabilities absent auth/policy/sandbox bypass; meanwhile at least one CVE entry claims “RCE via prompt injection” and is still awaiting analysis. This category merits careful validation before operational decisions. ²⁹
- **Trusted-proxy mode remains a high-risk configuration class.** Even after fixes, it inherently shifts security responsibility to proxy correctness and origin enforcement; more public, testable reference configurations would reduce repeated misdeployments. ¹²⁶
- **Marketplace scanning efficacy and bypass resistance** are uncertain: OpenClaw’s VirusTotal integration is explicitly “not a silver bullet,” and malicious skills can rely on natural language coercion rather than signatures. Empirical false-negative rates are unknown from primary sources. ⁹¹

- **Upgrade and drift risk is structural** due to near-daily releases and high advisory volume; organizations need automation for version compliance and configuration drift detection to stay meaningfully secure. ¹²⁷
- **Host isolation requirements for “hostile” inputs/users:** OpenClaw explicitly recommends splitting gateways by OS user/host for hostile-user isolation. Research question: what minimal isolation layer (containers vs VMs) is sufficient given the breadth of tool capability and secret handling? ¹²⁸

¹ ⁹ ¹⁹ ¹²⁵ <https://openclaw.ai/blog/introducing-openclaw>

<https://openclaw.ai/blog/introducing-openclaw>

² ⁵ ²² ⁷⁷ ⁸⁶ ⁸⁹ ¹²⁴ **Trust - OpenClaw**

<https://trust.openclaw.ai/>

³ ¹⁸ ³⁰ ³¹ ⁷⁹ ⁸⁸ ¹⁰² <https://github.com/openclaw/openclaw/security/advisories/GHSA-g8p2-7wf7-98mq>

<https://github.com/openclaw/openclaw/security/advisories/GHSA-g8p2-7wf7-98mq>

⁴ ²⁵ ⁵² ⁵³ ⁵⁶ ⁸² ⁹⁷ ¹¹³ ¹¹⁵ ¹¹⁸ <https://github.com/openclaw/openclaw/security/advisories/GHSA-5wcw-8jjv-m286>

<https://github.com/openclaw/openclaw/security/advisories/GHSA-5wcw-8jjv-m286>

⁶ ¹⁶ ⁷³ ⁷⁵ ⁸⁷ ⁹⁸ ¹⁰⁷ ¹¹⁴ <https://docs.openclaw.ai/tools/skills>

<https://docs.openclaw.ai/tools/skills>

⁷ ¹¹² **GitHub - openclaw/openclaw: Your own personal AI assistant. Any OS. Any Platform. The lobster way.**
· **GitHub**

<https://github.com/openclaw/openclaw>

⁸ ¹⁰ ¹¹ ¹⁷ ²⁹ ⁶⁵ ⁷² ¹¹⁰ ¹²⁸ <https://docs.openclaw.ai/gateway/security>

<https://docs.openclaw.ai/gateway/security>

¹² ⁸³ ⁹⁰ ⁹⁵ ⁹⁹ ¹⁰⁰ ¹⁰⁹ ¹¹⁹ <https://docs.openclaw.ai/web/control-ui>

<https://docs.openclaw.ai/web/control-ui>

¹³ ⁹⁴ ⁹⁶ ¹²⁰ ¹²¹ <https://docs.openclaw.ai/gateway/openresponses-http-api>

<https://docs.openclaw.ai/gateway/openresponses-http-api>

¹⁴ ⁷¹ ¹⁰⁵ ¹²² <https://docs.openclaw.ai/tools/multi-agent-sandbox-tools>

<https://docs.openclaw.ai/tools/multi-agent-sandbox-tools>

¹⁵ ¹⁰⁴ <https://docs.openclaw.ai/gateway/sandboxing>

<https://docs.openclaw.ai/gateway/sandboxing>

²⁰ **openclaw/CHANGELOG.md at main · openclaw/openclaw · GitHub**

<https://github.com/openclaw/openclaw/blob/main/CHANGELOG.md>

²¹ ¹¹⁷ ¹²⁷ **Internal Error**

²³ ⁹¹ ¹⁰⁸ **OpenClaw Partners with VirusTotal for Skill Security — OpenClaw Blog**

<https://openclaw.ai/blog/virustotal-partnership>

²⁴ ²⁶ ²⁸ ⁷⁶ ⁹³ <https://github.com/openclaw/openclaw/security/advisories>

<https://github.com/openclaw/openclaw/security/advisories>

27 <https://nvd.nist.gov/vuln/detail/CVE-2026-25253>
<https://nvd.nist.gov/vuln/detail/CVE-2026-25253>

32 33 <https://nvd.nist.gov/vuln/detail/CVE-2026-24763>
<https://nvd.nist.gov/vuln/detail/CVE-2026-24763>

34 <https://nvd.nist.gov/vuln/detail/CVE-2026-25157>
<https://nvd.nist.gov/vuln/detail/CVE-2026-25157>

35 36 80 <https://nvd.nist.gov/vuln/detail/CVE-2026-25593>
<https://nvd.nist.gov/vuln/detail/CVE-2026-25593>

37 38 81 <https://nvd.nist.gov/vuln/detail/CVE-2026-25475>
<https://nvd.nist.gov/vuln/detail/CVE-2026-25475>

39 <https://nvd.nist.gov/vuln/detail/CVE-2026-28472>
<https://nvd.nist.gov/vuln/detail/CVE-2026-28472>

40 41 42 <https://nvd.nist.gov/vuln/detail/CVE-2026-28470>
<https://nvd.nist.gov/vuln/detail/CVE-2026-28470>

43 <https://nvd.nist.gov/vuln/detail/CVE-2026-28363>
<https://nvd.nist.gov/vuln/detail/CVE-2026-28363>

44 61 106 <https://github.com/openclaw/openclaw/security/advisories/GHSA-3c6h-g97w-fg78>
<https://github.com/openclaw/openclaw/security/advisories/GHSA-3c6h-g97w-fg78>

45 <https://nvd.nist.gov/vuln/detail/CVE-2026-28485>
<https://nvd.nist.gov/vuln/detail/CVE-2026-28485>

46 48 <https://nvd.nist.gov/vuln/detail/CVE-2026-28458>
<https://nvd.nist.gov/vuln/detail/CVE-2026-28458>

47 <https://github.com/openclaw/openclaw/security/advisories/GHSA-mr32-vwc2-5j6h>
<https://github.com/openclaw/openclaw/security/advisories/GHSA-mr32-vwc2-5j6h>

49 50 51 62 85 92 <https://nvd.nist.gov/vuln/detail/CVE-2026-27488>
<https://nvd.nist.gov/vuln/detail/CVE-2026-27488>

54 55 <https://github.com/openclaw/openclaw/security/advisories/GHSA-rqpp-rjj8-7wv8>
<https://github.com/openclaw/openclaw/security/advisories/GHSA-rqpp-rjj8-7wv8>

57 <https://github.com/openclaw/openclaw/security/advisories/GHSA-v773-r54f-q32w>
<https://github.com/openclaw/openclaw/security/advisories/GHSA-v773-r54f-q32w>

58 70 101 <https://github.com/advisories/GHSA-jmmg-jqc7-5qf4>
<https://github.com/advisories/GHSA-jmmg-jqc7-5qf4>

59 78 123 <https://trust.openclaw.ai/trust/threatmodel>
<https://trust.openclaw.ai/trust/threatmodel>

60 69 <https://www.oasis.security/blog/openclaw-vulnerability>
<https://www.oasis.security/blog/openclaw-vulnerability>

63 <https://nvd.nist.gov/vuln/detail/CVE-2026-26972>
<https://nvd.nist.gov/vuln/detail/CVE-2026-26972>

64 116 <https://nvd.nist.gov/vuln/detail/CVE-2026-29609>

<https://nvd.nist.gov/vuln/detail/CVE-2026-29609>

66 84 **Run OpenClaw on Blaxel**

https://docs.blaxel.ai/Tutorials/OpenClaw?utm_source=chatgpt.com

67 **Clawdbot control UI can be exposed via default config ...**

https://github.com/openclaw/openclaw/issues/2245?utm_source=chatgpt.com

68 126 <https://docs.openclaw.ai/gateway/trusted-proxy-auth>

<https://docs.openclaw.ai/gateway/trusted-proxy-auth>

74 **OpenClaw's AI 'skill' extensions are a security nightmare**

https://www.theverge.com/news/874011/openclaw-ai-skill-clawhub-extensions-security-nightmare?utm_source=chatgpt.com

103 **Tools - OpenClaw**

https://docs.openclaw.ai/tools?utm_source=chatgpt.com

111 **Gateway Runbook**

https://docs.openclaw.ai/gateway?utm_source=chatgpt.com